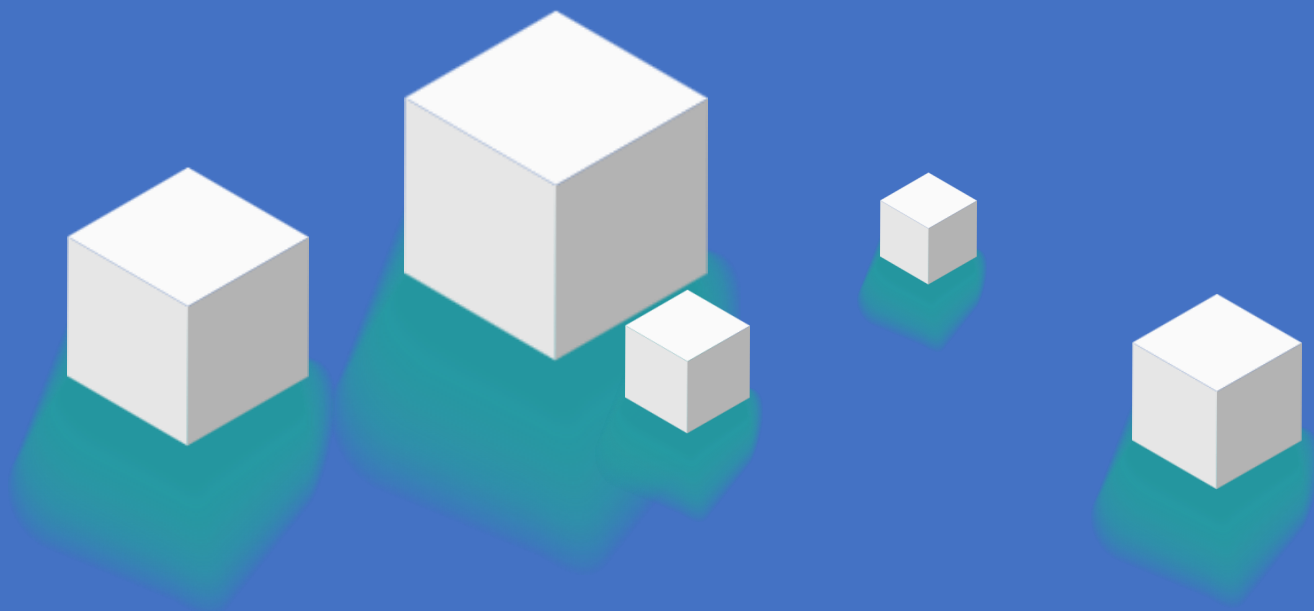


网格与多因子



>> 今天的学习目标

网格与多因子

- 经典网格交易策略

网格交易的四要素、交易原理、引擎逻辑

CASE: 经典网格策略

CASE: 缠论中枢网格策略

CASE: 中枢网格 + 趋势联动

- 多因子选股

从策略到选股

因子类型、因子评价、分层回测

CASE: 多因子评价框架

CASE: 多因子打分选股

CASE: 小市值轮动策略

CASE: 因子选股 + 中枢网格

CASE: ML增强多因子

经典网格交易策略

核心原理

Thinking:海龟和缠论教的是趋势市赚钱,但市场大部分时间在做什么?

70%的时间在震荡! 价格在一个区间内来回波动,既没有明确上涨也没有明确下跌。

海龟的突破策略在震荡市中频繁假突破 → 反复止损亏钱

缠论的三买信号在震荡市中很少出现 → 大部分时间空仓

网格策略就是为震荡市设计的收租模型:

把价格区间切成N个格子,每跌一格买一份,每涨一格卖一份。

不预测涨跌,只要价格在区间内波动就赚差价。

核心原理

维度	海龟交易	缠论量化	网格交易
核心哲学	突破 → 跟随趋势	结构 → 识别转折点	震荡 → 区间收租
适合行情	趋势市(单边涨/跌)	趋势转折点	震荡市(区间波动)
信号逻辑	价格突破N日高/低	分型/笔/中枢	价格穿越格子线
赚钱方式	大趋势的大利润	结构性突破	每格固定小利润
风险来源	假突破止损	背驰再背驰	单边下跌满仓被套

网格交易的四要素

Thinking: 网格策略需要回答哪四个问题?

1. 网格区间 — 在什么价格范围内做网格?
2. 格子数量 — 切几格? (越多交易越频繁, 每次赚越少)
3. 每格仓位 — 每穿越一格买/卖多少?
4. 出界处理 — 价格跌破下界(满仓被套) / 涨破上界(踏空)

网格交易的原理

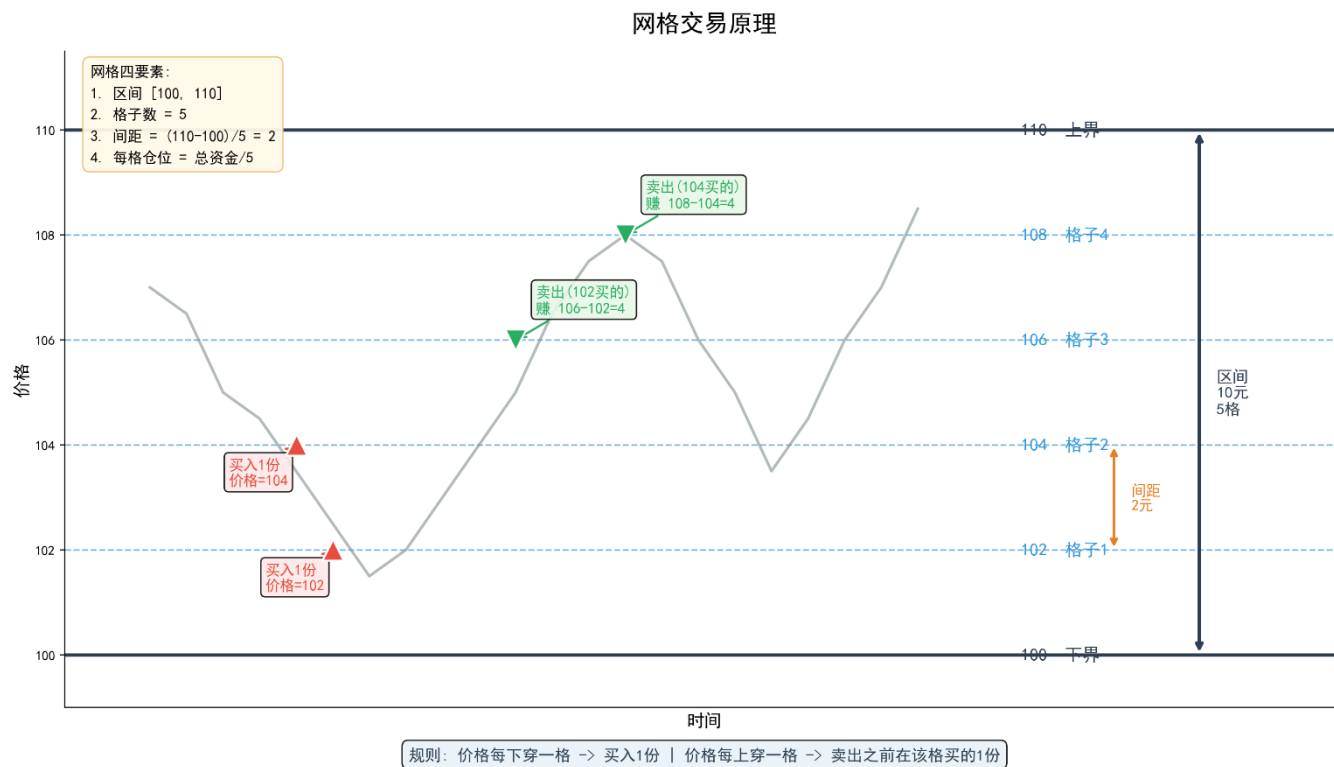
区间 [100, 110], 5格, 每格间距=2元

每格买卖赚固定差价 = $\text{grid_size} = (\text{上界} - \text{下界}) / \text{格子数}$

规则:

价格每下穿一格 => 买入一份

价格每上穿一个 => 卖出一份



网格引擎核心逻辑

```
class GridEngine:
    def __init__(self, upper, lower, num_grids, total_capital):
        self.grid_size = (upper - lower) / num_grids
        self.levels = [lower + i * self.grid_size for i in range(num_grids + 1)]
        self.position_at = [0] * (num_grids + 1) # 每格持仓状态

    def update(self, price):
        curr_cell = self.get_cell(price)
        if curr_cell < prev_cell:
            # 价格下跌, 穿越格子 → 买入
            for cell in range(prev_cell - 1, curr_cell - 1, -1):
                self.buy(cell)
        elif curr_cell > prev_cell:
            # 价格上涨, 穿越格子 → 卖出
            for cell in range(prev_cell, curr_cell):
                self.sell(cell)
```


CASE: 经典网格策略

CASE: 经典网格策略

TO DO: 在多只标的上运行固定网格, 对比不同市场环境的表现

网格参数:

- 网格区间: 过去60日最高/最低价 (+2%缓冲)
- 格子数量: 8格
- 每格仓位: 总资金的90% / 8 = 每格约11.25%
- 出界处理: 超出范围时停止买入/卖出, 等待价格回归

CASE: 经典网格策略

多标的回测结果 (2024-01 ~ 2025-12):

标的	网格收益	买入持有	胜率	夏普	交易次数
沪深300ETF	5.56%	46.10%	81.80%	1.86	11
贵州茅台	18.46%	-12.80%	85.70%	1.36	7
平安银行	5.65%	49.20%	87.50%	2.14	8
纳指ETF	6.31%	54.90%	64.30%	3.34	14

CASE: 经典网格策略

Thinking: 网格策略在茅台上赚了18%, 买入持有亏了12%。但在纳指ETF上, 网格只赚6%, 买入持有赚了54%。
为什么?

茅台2024-2025处于震荡市, 价格在1370~1650之间反复。网格反复做差价, 累计赚了18%。

纳指ETF2024-2025是单边上涨趋势。网格策略只能赚到区间内的小波动, 大趋势利润全部踏空。

=> 关键发现:

- 网格策略胜率极高(65%-88%), 但盈亏比低(每次赚固定一格)
- 震荡市: 网格稳定赚差价, 优于买入持有
- 趋势市: 网格容易踏空, 远差于买入持有
- 风险: 价格单边下跌时持续加仓 → 满仓被套

CASE: 经典网格策略

Thinking: 网格区间用 过去60日最高/最低价 来定, 这个方法有什么问题?

- 人为拍脑袋! 60日为什么不是30日或120日? 最高/最低价真的是合理的区间吗?
- 缠论给了一个更好的答案: 中枢的ZG/ZD就是市场自己"选出来"的震荡区间。

CASE：缠论中枢网格策略

CASE: 缠论中枢网格策略

Thinking: 什么是中枢?

- 中枢 = 至少3笔价格区间有重叠 → 价格在此区间内反复震荡
- ZG(中枢高点) = 各笔高点中的最小值
- ZD(中枢低点) = 各笔低点中的最大值

=> 中枢就是市场自己选出来的震荡区间

=> 网格策略理想的执行环境

CASE: 缠论中枢网格策略

维度	固定网格	中枢网格
区间来源	人为设定(60日高低点)	市场结构(缠论中枢ZG/ZD)
区间合理性	可能偏大或偏小	由3笔重叠定义, 有结构基础
出界处理	停止交易等待	停止网格, 中枢切换后重建
适应性	固定不变	新中枢形成自动重建

CASE: 缠论中枢网格策略

中枢网格引擎

```
class ChanGridEngine(GridEngine):
    """以缠论中枢的 ZG/ZD 作为网格边界"""
    def __init__(self, zg, zd, num_grids=6, total_capital=0):
        super().__init__(upper=zg, lower=zd, ...)
        self.zg = zg
        self.zd = zd

    def switch_zhongshu(self, new_zg, new_zd):
        """切换到新中枢, 重建网格"""
        self.zg, self.zd = new_zg, new_zd
        self.levels = [new_zd + i * gs for i in range(num_grids + 1)]
        self.position_at = [0] * (num_grids + 1)

    def deactivate(self):
        """价格突破中枢时停用网格"""
        self.active = False
```

CASE: 缠论中枢网格策略

策略逻辑

```
class ChanGridStrategy(bt.Strategy):  
    def next(self):  
        zg = self.data.chan_zg[0] # 当前中枢上沿  
        zd = self.data.chan_zd[0] # 当前中枢下沿  
        price = self.data.close[0]  
  
        # 1. 中枢切换 → 重建网格  
        if zg != self.current_zg or zd != self.current_zd:  
            self.grid.switch_zhongshu(zg, zd)  
  
        # 2. 价格在中枢内 → 网格交易  
        if zd <= price <= zg:  
            signals = self.grid.update(price)  
            for sig in signals:  
                self.execute(sig)
```

```
# 3. 价格突破中枢 → 停止网格, 清仓  
if price > zg * 1.005 or price < zd * 0.995:  
    self.close()  
    self.grid.deactivate()
```

CASE: 缠论中枢网格策略

数据流:

原始K线 → run_chan() → chan_to_signal_df() → ChanPandasData → Backtrader策略



识别中枢
识别买卖点



生成chan_zg/zd
生成chan_signal



自定义数据源
含缠论信号线

chanpy_wrapper.py → data_loader.py → 2-缠论中枢网格策略.py

CASE: 缠论中枢网格策略

TO DO: 用 chan.py 分析中枢, 以 ZG/ZD 作为网格边界进行回测

以贵州茅台为例 (2023-01 ~ 2025-12):

chan.py 分析结果: 38笔, 3个中枢, 6个买卖点

中枢	时间	ZG	ZD	宽度	宽度%
1	2023-03 ~ 2023-12	1631.36	1572.56	58.8	3.70%
2	2024-06 ~ 2024-08	1437.5	1338.5	99	7.10%
3	2024-10 ~ 2025-02	1591.6	1413.37	178.23	11.90%

CASE: 缠论中枢网格策略

Thinking: 3个中枢的宽度从3.7%扩大到11.9%, 这意味着什么?

中枢1(3.7%宽): 多空分歧小, 价格在很窄的区间内震荡

中枢3(11.9%宽): 多空分歧大, 价格波动剧烈

对网格策略的影响:

- 窄中枢: 每格间距小, 交易频繁但每次赚得少
- 宽中枢: 每格间距大, 交易少但每次赚得多

Summary

三策略对比 (贵州茅台 2023-2025):

指标	买入持有	固定网格	中枢网格
总收益	-12.76%	13.16%	1.62%
最大回撤	—	18.42%	1.61%
夏普比率	—	0.78	-2.54
交易次数	—	4	2
胜率	—	75.00%	100.00%

Summary

Thinking: 中枢网格只赚了1.62%, 远不如固定网格的13.16%。中枢网格有什么优势?

看最大回撤! 中枢网格仅1.61%, 固定网格18.42%。

中枢网格的核心价值是风险控制:

- 只在中枢内交易 → 区间有结构支撑
- 出中枢即停止 → 不在趋势中逆势加仓
- 代价是收益较低 → 大部分时间在等待

固定网格的问题: 价格单边下跌时持续加仓, 可能满仓被套(回撤18.42%)

中枢网格的解决: 跌破ZD就停止, 最多亏中枢内的仓位

Summary

多标的横向对比

标的	中枢数	中枢网格收益	固定网格收益	买入持有
贵州茅台	3	1.62%	13.16%	-12.76%
中芯国际	3	-0.82%	24.47%	195.69%
平安银行	4	1.27%	3.70%	-4.28%
纳指ETF	1	0.00%	5.44%	135.00%

Summary

Thinking: 纳指ETF只有1个中枢, 中枢网格收益0%。中芯国际有3个中枢但也亏了。中枢多少会影响策略效果吗?

- 中枢越多 → 震荡越多 → 网格机会越多
- 中枢越少 → 趋势越强 → 网格不适用
- 纳指ETF: 趋势股, 1个中枢, 网格几乎无用
- 平安银行: 4个中枢, 震荡较多, 网格有+1.27%正收益

=> 关键发现:

- 中枢网格的区间由市场结构决定, 比固定区间更精准
- 中枢多的标的(震荡多): 网格机会更多, 收益更稳
- 中枢少的标的(趋势强): 网格机会少, 但出中枢停止避免大亏
- 中枢切换时自动重建网格, 适应市场变化

CASE: 中枢网格 + 趋势联动

CASE: 中枢网格 + 趋势联动

Thinking: 中枢网格在出中枢后就停止交易等待 — 这浪费了趋势行情。能不能两边都赚？

自适应策略的进化版:

- ADX判断趋势/震荡 → 切换MACD/RSI
- 缠论中枢判断趋势/震荡 → 切换网格/趋势跟踪

模式	触发条件	交易逻辑	目标
GRID	价格在中枢内 [ZD, ZG]	网格低买高卖	赚震荡差价
TREND_UP	价格向上突破 ZG	ATR跟踪止损持仓	赚趋势利润
WAIT	价格向下跌破 ZD	空仓等待新中枢	回避下跌

CASE: 中枢网格 + 趋势联动

策略核心代码

```
class ChanHybridStrategy(bt.Strategy):
    def next(self):
        # GRID模式: 中枢内做网格
        if self.mode == 'GRID':
            if price > zg * 1.005: # 向上突破
                self.close()      # 清网格仓位
                self.buy(size=...) # 趋势入场
                self.trail_stop = price - 2.5 * ATR
                self.mode = 'TREND_UP'
            elif price < zd * 0.995: # 向下跌破
                self.close()      # 清仓防守
                self.mode = 'WAIT'
        else:
            self.grid.update(price) # 正常网格
```

```
# TREND_UP模式: ATR跟踪止损
if self.mode == 'TREND_UP':
    self.trail_stop = max(self.trail_stop,
                           highest - 2.5 * ATR)
    if price < self.trail_stop:
        self.close()
        self.mode = 'WAIT'
```

CASE: 中枢网格 + 趋势联动

TO DO: 对比纯中枢网格 vs 中枢网格+趋势联动

以贵州茅台为例, 模式切换记录:

日期	从	到	价格
2023/3/20	WAIT	GRID	1578.19
2023/3/30	GRID	TREND_UP	1648.59
2023/4/12	TREND_UP	WAIT	1542.69
2024/6/24	WAIT	GRID	1401.04
2024/10/8	WAIT	GRID	1647.49
2025/1/6	GRID	WAIT	1388.37

模式切换由缠论结构驱动, 不是人为判断。

CASE: 中枢网格 + 趋势联动

Thinking: 混合策略在趋势跟踪部分反而亏了钱。这说明什么?

不是每个增强都一定有效! 趋势跟踪的效果取决于:

- 出中枢后趋势是否延续 (很多时候是假突破)
- ATR跟踪止损的参数是否合适
- 标的本身是否有强趋势特征

CASE: 中枢网格 + 趋势联动

Thinking: 那混合策略没有用吗?

在趋势明确的标的(如纳指ETF)上, 趋势跟踪模式能捕捉到大行情。

在震荡为主的标的(如茅台)上, 纯中枢网格更稳健。

策略选择矩阵:

标的特征	推荐策略	原因
高震荡、低趋势	纯中枢网格	中枢多, 网格机会多
有趋势、有震荡	混合策略	两种行情都能赚
强趋势	海龟/缠论三买	网格在趋势市没优势

Summary

Thinking: 网格策略能做到什么?

- 震荡市稳定收租: 只要价格在区间内波动, 就有差价利润
- 胜率极高: 70%-90%, 远高于趋势策略
- 不预测方向: 无需判断涨跌, 纪律性执行
- 中枢网格: 让区间边界由市场结构决定, 比人为设定更科学
- 出中枢即停: 避免趋势市中逆势加仓的灾难性亏损

Summary

Thinking: 网格策略做不到什么?

- 单边行情赚大钱: 趋势利润必然踏空
- 避免满仓被套: 如果价格持续跌破下界, 网格持仓会越来越重
- 适合所有标的: 趋势型股票(纳指ETF)不适合网格
- 保证绝对收益: 网格赚的是相对收益(差价), 不是绝对收益(方向)

Summary

Thinking: 网格策略的效果高度依赖标的选择。

- 震荡股(茅台): 网格表现好
- 趋势股(纳指ETF): 网格几乎无用

=> 能不能用科学的方法, 从全市场选出 适合做网格的 标的?

=> 多因子选股 要解决的问题。

打卡：网格交易策略



针对你的自选股，搭建网格交易策略

- 使用基础缠论方
- 使用固定网格策略
- 用 `chan.py` 分析中枢, 运行中枢网格策略
- 对比固定网格 vs 中枢网格 vs 买入持有

在2025年，你的自选股，通过网格交易策略，会取得怎样的收益？

你的自选股适合做网格吗？(看ADX和中枢数量)

CASE: 多因子选股

从策略到选股

Thinking: 之前的策略都是 给什么标的用什么标的。如何科学地从全市场选出值得交易的品种?

海龟策略/缠论策略/网格策略, 效果高度依赖标的的选择。

=> 需要一个系统化的选股方法。

因子(Factor) = 能预测未来收益的指标

多因子选股 = 综合多个因子信息, 选出最优股票组合

因子类型

Thinking：因子类型都有哪些？

因子类型	例子	计算工具	含义
动量因子	ROC(20), ROC(60)	TA-Lib	过去N日涨幅, 强者恒强
波动率因子	ATR/Close	TA-Lib	价格波动幅度
超买超卖	RSI(14)	TA-Lib	短期涨跌过度
趋势强度	ADX(14)	TA-Lib	当前是趋势市还是震荡市
量能因子	当日量/20日均量	TA-Lib	资金活跃程度
动能因子	MACD柱状图	TA-Lib	多空力量对比

因子评价

Thinking: 我算了一个因子, 怎么知道它能不能预测未来收益?

两种方法:

1. IC分析 — 因子值与未来收益的相关性
2. 分层回测 — 按因子值分组, 看各组收益差异

IC (信息系数)

IC = Spearman Rank Correlation(因子值, 未来N日收益)

每期(月末)计算一次:

- 1. 截面上所有股票的因子值排名
- 2. 同一批股票的未来20日收益排名
- 3. 两组排名的Spearman相关系数 = 当期IC

指标	含义	判定标准
IC均值	因子预测能力的方向和强度	$ IC > 0.05$ 有效
IC标准差	因子有效性的稳定程度	越小越稳定
$ICIR = IC均值 / IC标准差$	综合评价	> 0.5 优秀, $0.3 \sim 0.5$ 良好
IC > 0 比例	因子有效的时间占比	$> 60\%$ 较好

IC (信息系数)

Thinking: 为什么用Spearman排名相关, 而不是Pearson?

- 因为我们关心的是因子值排名高的股票, 未来收益排名也高。
- Spearman只看排名顺序, 不受极端值影响, 更适合金融数据。

分层回测

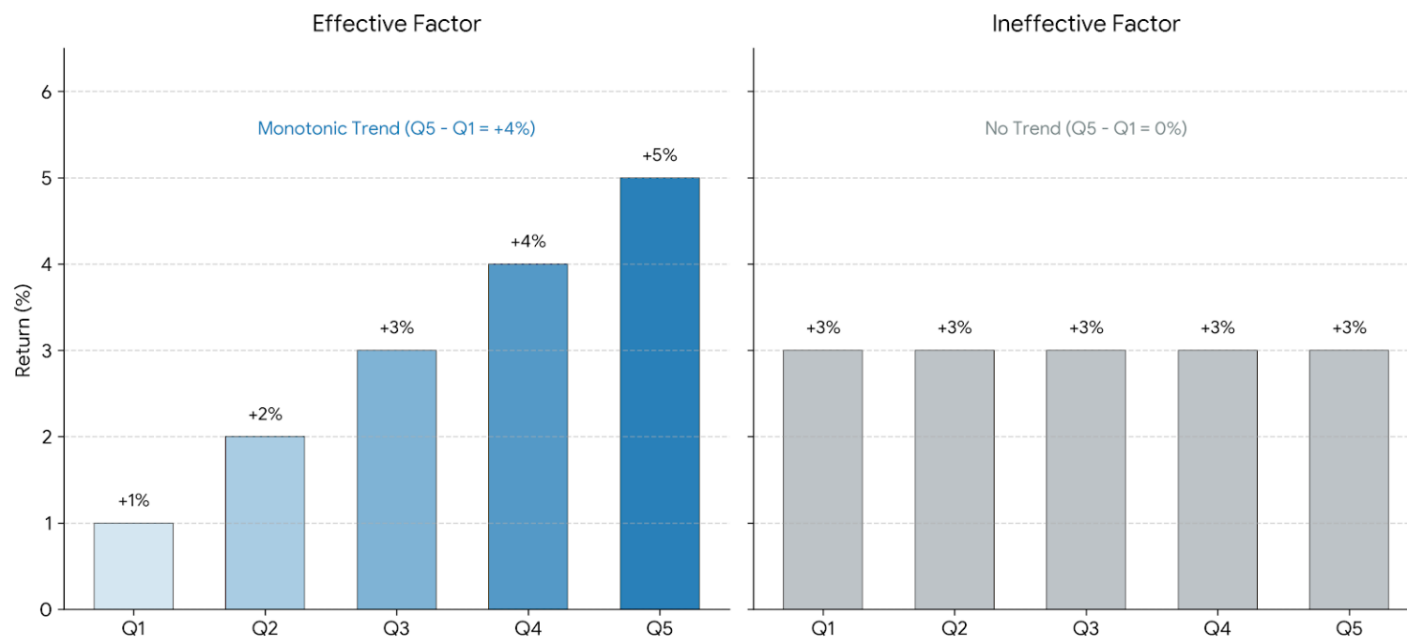
分层回测：

按因子值从小到大分成5组：

- Q1: 因子值最小的20%股票
- Q2~Q4: 中间组
- Q5: 因子值最大的20%股票

每月末重新分组, 计算各组平均收益。

如果Q5-Q1价差显著 → 因子有区分能力



CASE：多因子评价框架

TO DO: 对全市场6900+只标的, 计算技术因子的IC和分层回测

IC分析结果:

数据: 6974只标的, 2024-2025

因子	IC均值	IC标准差	ICIR	判定
volatility(波动率)	-0.07	0.2071	-0.3383	良好, 反向
momentum_20d(动量)	-0.0077	0.1852	-0.0415	较弱
rsi_14	-0.001	0.1977	-0.0052	较弱

CASE：多因子评价框架

Thinking: 波动率因子的IC为负(-0.07), ICIR=-0.34。这意味着什么?

IC为负 = 波动率越高, 未来收益越低 → 低波动股票表现更好!

ICIR=-0.34 → 这个关系比较稳定(良好级别)

A股的低波动异象: 低波动股票长期跑赢高波动股票, 与经典金融理论(高风险高回报)矛盾。

CASE：多因子评价框架

分层回测结果

波动率因子的5分位回测:

分组	平均月收益	累计收益
Q1(最低波动)	2.21%	50.37%
Q2	2.75%	64.82%
Q3	3.56%	90.41%
Q4	3.83%	97.36%
Q5(最高波动)	3.13%	69.46%

Thinking: Q4(较高波动)收益最高, 不是Q1(最低波动)? 这和IC分析矛盾吗?

不矛盾! IC是线性相关, 分层回测是非线性的。
波动率和收益是倒U型关系: 适度波动最好, 极高和极低都不好。
Q5(极高波动)包含很多问题股, 拉低了平均收益。

CASE: 多因子打分选股

从因子到策略

Thinking: 单个因子的预测能力有限(ICIR大多 < 0.5)。怎么办?

多因子组合! 把多个维度的信息综合起来, 比单因子更稳健。

8因子打分体系:

因子	权重	方向	含义
momentum_20d	20%	正向	短期动量, 强者恒强
momentum_60d	15%	正向	中期动量
volatility	15%	反向	低波动优先
rsi_14	10%	反向	超卖区间更优
adx_14	10%	正向	趋势越强越好
turnover_ratio	10%	正向	量能放大信号
price_position	10%	反向	价格在区间越低越好
macd_signal	10%	正向	MACD柱状图 > 0

CASE：多因子评价框架

打分流程

```
def score_stocks(factor_df):  
    for fname, cfg in FACTOR_CONFIG.items():  
        # 1. 横截面排名归一化 (0~1)  
        rank = factor_df[fname].rank(pct=True)  
  
        # 2. 反向因子翻转  
        if cfg['direction'] < 0:  
            rank = 1 - rank  
  
        # 3. 按权重加权求和  
        factor_df['score'] += rank * cfg['weight']  
  
    return factor_df.sort_values('score', ascending=False)
```


月度调仓回测

每月末:

1. 对全市场计算8个因子
2. 横截面排名打分
3. 选出 Top-N 股票
4. 等权重配置, 持有至下月末

月末T → 计算因子 → 打分排名 → 选Top-N → 等权持有 → 月末T+1



月末T+1 → 重新计算 → 重新打分 → 调仓 → 等权持有 → 月末T+2

月度调仓回测

TO DO: 不同 Top-N 的多因子选股回测

回测结果 (2024-2025, 6974只标的):

Top-N	总收益	年化	最大回撤	夏普	月度胜率
Top-5	19.25%	10.12%	22.41%	0.49	63.20%
Top-10	18.88%	9.93%	12.37%	0.56	63.20%
Top-20	7.76%	4.18%	18.44%	0.25	52.60%

CASE: 多因子打分选股

Thinking: Top-5收益最高(19.25%), 但Top-10的夏普比率更高(0.56 vs 0.49)。怎么选?

- Top-5: 集中度高, 弹性大, 但单只股票对组合影响大 → 波动大(回撤22%)
- Top-10: 适度分散, 收益和风险平衡 → 回撤仅12%
- Top-20: 过度分散, 因子效果被稀释 → 收益下降到7.76%

=> 结论: Top-10是最佳平衡点 — 夏普最高, 意味着每承担1单位风险获得的收益最多。

CASE: 多因子打分选股

Thinking: 多因子打分全部用的是技术因子(TA-Lib计算)。如果加入基本面因子(PE/ROE)呢?

这是因子研究的方向:

- 基本面因子(PE/ROE/毛利率): 需要财务数据表
- 市场微结构因子(买卖价差/订单流): 需要高频数据
- 事件因子(财报日/股东增减持): 需要公告数据
- 另类因子(舆情/分析师评级): 需要AI大模型处理

=> 先聚焦技术因子, 生产级系统可以加入更多维度。

CASE: 小市值轮动策略

小市值效应

Thinking: A股有一个长期有效的规律: 小市值股票组的长期收益 > 大市值股票组。这叫什么?

SMB (Small Minus Big) — Fama-French三因子模型中的市值因子。

因子	含义	A股有效性
市场因子(MKT)	大盘涨跌	基本有效
市值因子(SMB)	小盘 - 大盘	长期有效, 但近年减弱
价值因子(HML)	低PE - 高PE	阶段性有效

CASE: 小市值轮动策略

纯小市值策略:

1. 每月末按市值排序
2. 选出市值最小的N只
3. 等权持有至下月末

增强版:

1. 先选市值最小的50只(候选池)
2. 在候选池中过滤: 排除20日动量 $< -20\%$ (暴跌股), 排除波动率 $> 5\%$ (极端波动股)
3. 按 市值排名 + 正动量 + 低波动 打分
4. 选出Top-N

CASE: 小市值轮动策略

```
def rank_enhanced(all_data, calc_date, top_n=20, pool_size=50):  
    # 第一步: 市值排名选候选池  
    candidates = sort_by_market_cap()[:pool_size]  
  
    # 第二步: 过滤垃圾股  
    for code in candidates:  
        if momentum_20d < -20: continue # 排除暴跌股  
        if volatility > 0.05: continue # 排除极端波动  
  
    # 第三步: 综合打分  
    score = cap_rank * 0.5 + momentum * 0.3 + low_vol * 0.2
```


CASE: 小市值轮动策略

TO DO: 对比纯市值排序 vs 市值+动量+波动率过滤

回测结果 (2024-2025, Top-10):

指标	纯小市值	增强版
总收益	15.93%	10.07%
年化收益	8.43%	5.40%
最大回撤	2.47%	1.68%
夏普比率	1.12	0.89
卡玛比率	3.41	3.21
月度胜率	63.60%	63.20%

CASE: 小市值轮动策略

Thinking: 纯小市值收益更高(15.93% vs 10.07%), 增强版的价值在哪?

看最大回撤: 增强版仅1.68%, 纯小市值2.47%。

纯小市值的风险:

- 可能选到ST股、退市风险股
- 可能选到暴跌中的股票(越跌市值越小, 越容易被选中)
- 没有过滤 = 什么都买

增强版的价值:

- 动量过滤: 排除正在暴跌的小票(止血)
- 波动率过滤: 排除极端波动的垃圾股(避雷)
- 市值 + 动量 + 波动率 = 最基础的多因子框架

CASE: 小市值轮动策略

不同 Top-N 对比

Top-N	总收益	年化	最大回撤	夏普	胜率
Top-5	5.77%	3.12%	1.52%	0.41	63.20%
Top-10	10.07%	5.40%	1.68%	0.89	63.20%
Top-20	12.96%	6.90%	2.99%	0.93	57.90%

Thinking: 小市值策略中, Top-20反而收益最高(12.96%)。和多因子打分(Top-5最高)正好相反。为什么?

因为小市值效应是群体性的, 不是个股性的。

- 多因子打分: 选的是 个股质量最好的前N只, 越集中越好
- 小市值轮动: 选的是一群小票, 越分散效应越稳定
- Top-20覆盖更多小票, 捕获市值效应更充分

Summary

三种选股方法对比

方法	逻辑	优势	劣势
多因子打分	8因子加权求和	综合全面	权重人为设定
小市值轮动	按市值排序	简单有效	容易选到垃圾股
增强小市值	市值+动量+波动率	过滤垃圾股	可能过度过滤

Summary

因子选股的核心价值：

信号 → 因子 → 打分 → 排名 → 选股 → 调仓

- 因子是量化投资的基础语言
- IC/ICIR是评价因子好坏的金标准
- 分层回测验证因子的区分能力
- 多因子组合比单因子更稳健
- 选股频率: 月度调仓 = 低换手率 + 低交易成本

Summary

Thinking: 现在有了两条线:

- 网格线: 固定网格 → 中枢网格 → 趋势联动
- 选股线: 因子评价 → 多因子打分 → 小市值轮动

=> 能不能把两条线融合? 用因子选出适合做网格的标的, 然后在上面做中枢网格?

打卡：多因子选股



针对你的股票池（也可以是全量的大A股票），进行多因子选股：

- 计算技术因子, 分析IC值
- 用8因子打分体系对全市场股票排名
- 运行月度调仓回测, 选择最佳的 Top-N
- 对比多因子选股 vs 小市值轮动的效果

你的自选股在因子打分中排名如何？

CASE: 因子选股 + 中枢网格

因子选股 + 中枢网格

Thinking: 之前学了两条独立的线:

网格线: 固定网格 → 中枢网格 → 趋势联动 (怎么交易)

选股线: 因子评价 → 多因子打分 → 小市值轮动 (交易什么)

=> 现在把它们融合: 用因子选出 适合做网格 的标的, 然后在上面做中枢网格。

因子选股 + 中枢网格

Thinking:什么标的适合做网格?

特征	适合网格	不适合网格	判断指标
趋势性	低(震荡为主)	高(单边趋势)	ADX低 = 好
波动率	适中(有格子差价)	太低或太高	ATR/Close
RSI	接近50(无方向)	极端值	RSI中性 = 好
动量	接近0(无趋势)	绝对值大	ROC小 = 好
中枢	有中枢(震荡区间)	无中枢	chan.py分析

网格适配因子配置

和多因子打分不同, 这里的因子方向和权重为网格优化:

因子	权重	方向	含义
ADX(14)	30%	反向	ADX越低 = 越震荡 = 越适合网格
波动率	25%	正向	适度波动有利于网格差价
RSI(14)	15%	中性	距离50越近越好(无方向)
20日动量	15%	中性	绝对值越小越好(无趋势)
换手率	15%	正向	足够的交易量保证流动性

网格适配因子配置

Thinking: 为什么ADX的权重最高(30%)?

因为ADX是最直接的 趋势/震荡 判断指标:

- $ADX < 20$: 震荡市, 网格天堂
- $ADX 20 \sim 40$: 趋势形成中, 网格风险增大
- $ADX > 40$: 强趋势, 网格必死

网格策略的前提就是 市场在震荡, 而ADX直接衡量这个前提是否成立。

CASE：因子选股 + 中枢网格

RSI和动量的特殊处理

```
def score_for_grid(factor_df):  
    for fname, cfg in GRID_FACTOR_CONFIG.items():  
        if cfg['direction'] == 0:  
            # 特殊因子: 距离中性值越近越好  
            if fname == 'rsi_14':  
                dist = (result[fname] - 50).abs() # RSI距离50  
                rank = 1 - dist.rank(pct=True) # 越近得分越高  
            elif fname == 'momentum_20d':  
                dist = result[fname].abs() # 动量绝对值  
                rank = 1 - dist.rank(pct=True) # 越小得分越高
```

Thinking: 为什么RSI不是 越低越好, 而是 越接近50越好?

在多因子选股中, RSI低(超卖)可能意味着反弹机会 → 反向因子

在网格选股中, RSI = 50意味着多空平衡 → 最适合震荡 → 中性因子

=> 同一个指标, 在不同策略目标下, 方向可以完全不同!

CASE：因子选股 + 中枢网格

融合策略流程

全市场 → 网格适配因子打分 → Top-N → chan.py分析 → 有中枢?

↓ Yes

中枢网格回测

↓ No

跳过该标的

CASE：因子选股 + 中枢网格

TO DO: 用网格适配因子打分选出最适合做网格的标的, 运行中枢网格回测

网格适配评分排名 (6852只标的):

排名	代码	得分	ADX	波动率	RSI	20D动量
1	301617.SZ	0.9305	11.1	0.0535	51.2	0.76%
2	301013.SZ	0.8864	10.2	0.0486	48.4	-2.23%
3	300591.SZ	0.8837	13.5	0.0589	50	1.95%
4	688095.SH	0.8718	14.9	0.0446	50.9	0.34%
5	603001.SH	0.8638	11.7	0.0435	54.3	-0.56%

CASE：因子选股 + 中枢网格

Thinking: 排名第一的 301617.SZ, ADX=11.1, RSI=51.2, 动量+0.76%。这些数字说明了什么?

- ADX=11.1: 远低于20, 典型的震荡市
- RSI=51.2: 几乎正好在50, 多空完美平衡
- 动量+0.76%: 接近0, 没有方向

=> 这只股票 没有方向, 价格在一个区间内来回晃 — 正是网格策略的理想猎物。

CASE：因子选股 + 中枢网格

Top-5 中枢网格回测：

代码	得分	中枢数	网格收益	买持收益	最大回撤	胜率
301617.SZ	0.9305	1	0.34%	27.62%	0.15%	100%
301013.SZ	0.8864	5	-0.55%	219.75%	0.77%	0%
300591.SZ	0.8837	4	-2.59%	103.70%	5.66%	50%
688095.SH	0.8718	3	-0.82%	93.35%	0.82%	0%
603001.SH	0.8638	4	10.41%	44.59%	5.77%	80%

组合平均: 收益=+1.36%, 最大回撤=2.64%

CASE：因子选股 + 中枢网格

Thinking: 603001.SH表现最好(+10.41%, 胜率80%), 得分排第5。得分最高的不一定收益最好?

因子打分是 事前预测, 回测收益是事后结果。

得分高 = 当前最像震荡市的股票, 但未来可能转为趋势。

603001.SH的特点:

- 4个中枢(震荡机会多)
- ADX=11.7(确实在震荡)
- 波动率适中(每格有足够差价)

301013.SZ得分第2但亏了:

- 虽然因子显示震荡, 但实际走出了大趋势(+219%)
- 网格策略踏空了趋势 → 验证了"网格不适合趋势股"

Summary

关键发现:

- 因子选股筛出 适合网格 的标的 → 提升网格策略效果
- 低ADX的股票在中枢内震荡更多 → 网格交易机会更多
- 组合平均回撤仅2.64% → 风险控制优秀
- 因子选股 + 缠论中枢 + 网格交易 = 完整的量化策略闭环

CASE: ML增强多因子

CASE：ML增强多因子

Thinking: 多因子打分(脚本5)用的是人工定权重 — 动量20%, 波动率15% ... 这些权重最优吗?

不一定! 人工权重有几个问题:

- 凭经验拍脑袋, 不是最优解
- 不同市场环境下最优权重可能不同
- 忽略了因子之间的交互关系(比如 高动量+低波动 可能比单个因子更有效)

=> ML的优势: 让模型从数据中学习最优的因子组合方式。

方法	权重来源	优势	劣势
等权打分	人工设定	简单直觉	可能不最优
IC加权	IC值决定	有理论依据	假设线性关系
LightGBM	数据学习	发现非线性组合	需要足够样本

CASE: ML增强多因子

特征工程: 13维特征

```
FEATURE_NAMES = [  
    # 动量系列 (4个)  
    'momentum_5d', 'momentum_10d', 'momentum_20d', 'momentum_60d',  
    # 波动/趋势 (3个)  
    'volatility', 'rsi_14', 'adx_14',  
    # 动能/量能 (3个)  
    'macd_hist', 'obv_slope', 'vol_ratio',  
    # 价格位置 (1个)  
    'price_position',  
  
    # 交互特征 (2个) ← ML独有  
    'mom_vol_cross', # 动量 * 波动率  
    'adx_rsi_cross', # ADX * (RSI-50)  
]
```

Thinking: 为什么要加交互特征(mom_vol_cross, adx_rsi_cross)?

因为因子之间的组合效应可能比单因子更强:

- 高动量 + 低波动: 稳定上涨, 质量最好的趋势
- 高动量 + 高波动: 暴涨暴跌, 不确定性大
- 高ADX + RSI>50: 确定性上涨趋势
- 高ADX + RSI<50: 确定性下跌趋势

交互特征让模型直接看到这些组合关系, 提升预测效果。

和缠论ML的思路一致: 海龟ML用突破特征, 缠论ML用结构特征, 多因子ML用因子交互特征。

CASE: ML增强多因子

Walk-Forward 滚动训练

- 训练窗口(12个月): 2023-01 ~ 2023-12, 预测: 2024-01
- 训练窗口(12个月): 2023-02 ~ 2024-01, 预测: 2024-02
- ... 滚动前进
- 训练窗口(12月): 2024-12~2025-11, 预测: 2025-12

每月:

1. 用过去12个月的数据训练LightGBM
2. 预测当月所有股票的未来20日收益
3. 选出预测值最高的Top-10
4. 用真实收益计算组合表现

CASE: ML增强多因子

Thinking: 为什么不用全部历史数据训练, 而是只用过去12个月?

避免 未来信息泄露! 如果用2024-2025全部数据训练, 再回头测2024年, 等于拿答案做题。

Walk-Forward保证:

- 训练数据 < 预测时点 (时间上不交叉)
- 模型定期更新 (适应市场变化)
- 和缠论ML(脚本8-ML增强缠论策略)完全相同的验证方法

CASE: ML增强多因子

LightGBM 模型配置

```
model = lgb.LGBMRegressor(  
    n_estimators=100, # 100棵树  
    max_depth=5,     # 最大深度5 (防过拟合)  
    learning_rate=0.05, # 学习率  
    subsample=0.8,    # 每棵树用80%样本  
    colsample_bytree=0.8, # 每棵树用80%特征  
    min_child_samples=10, # 叶节点最少10个样本  
)
```

Thinking: 为什么用LightGBM而不是随机森林或深度学习?

LightGBM的优势:

- 速度快: 6900只股票 * 31个月 = 19万样本, 训练只需几秒
- 处理表格数据: 因子数据本质是表格, 树模型比神经网络更适合
- 特征重要性: 自动输出每个因子的重要性排名
- 正则化: 自带L1/L2正则和树深度限制, 不容易过拟合

=> 和海龟ML/缠论ML用的是同一个模型框架。

CASE：ML增强多因子

TO DO: LightGBM 滚动训练 + 预测选股 vs 等权动量排序

训练集: 198,118条样本, 31个月, 6870只股票

ML vs 基准对比:

指标	动量排序(基准)	ML选股(LightGBM)
总收益	-88.50%	37.46%
年化收益	-57.95%	23.72%
最大回撤	91.94%	17.15%
夏普比率	-1.53	0.72
卡玛比率	-0.63	1.38
月度胜率	29.00%	52.60%

CASE: ML增强多因子

Thinking: 动量排序策略亏了88%! 但ML选股赚了37%。差距这么大?

动量排序的问题: 只看20日动量最高的10只 → 追涨杀跌

- 2024年某些月份大涨, 动量最高的股票已经涨到顶部
- 下个月回落暴跌, 反复追高被套

ML模型的优势: 综合13个特征, 不是简单追高

- 模型发现 高动量+低波动 才是好组合
- 模型发现 高动量+高波动 是陷阱(看似强势, 实则暴涨暴跌)
- 交互特征让模型看到了人类难以直觉判断的组合关系

CASE：ML增强多因子

最近一轮LightGBM模型的特征重要性:

特征	重要性
volatility(波动率)	314
momentum_60d(60日动量)	309
price_position(价格位置)	249
macd_hist(MACD动能)	247
mom_vol_cross(动量*波动率)	234
vol_ratio(量比)	218
adx_14(趋势强度)	214
momentum_10d	210

Thinking: 波动率排第1, 动量*波动率排第5。

和缠论ML的特征重要性有什么不同?

排名	多因子ML	缠论ML(三买)	海龟ML(突破)
1	波动率	中枢高度	波动率变化
2	60日动量	量比	趋势强度(ADX)
3	价格位置	MACD动能	盘整天数
4	MACD动能	RSI	波动率
5	动量*波动率	波动率	RSI

- 波动率/ATR在所有ML策略中都很重要 → 波动率是市场本质的特征之一
- 缠论ML独有 中枢高度 → 结构信息是缠论的核心价值
- 多因子ML的 动量*波动率 交互排第5 → 因子组合比单因子更有信息量
- 海龟ML重视 盘整天数下 → 突破策略关心盘整多久

Summary

ML选股的注意事项

风险点	说明	应对
过拟合	样本不足时模型记住噪音	max_depth限制, subsample, 滚动验证
未来泄露	用未来数据训练	Walk-Forward严格时间分割
因子失效	某些因子在特定时期无效	多因子分散, 定期重训练
数据质量	停牌/退市/财务造假	数据清洗, 极端值过滤
交易成本	月度调仓的交易费用	换手率控制, 冲击成本估算

Summary

网格线:

- 1-经典网格策略.py 固定区间, 等距切格, 机械执行
- 2-缠论中枢网格.py ZG/ZD作为网格边界, 市场结构驱动
- 3-中枢网格+趋势联动.py 中枢内网格, 出中枢趋势跟踪



选股线:

- 4-多因子评价框架.py IC/ICIR评价因子有效性
- 5-多因子打分选股.py 8因子打分, Top-N月度调仓
- 6-小市值轮动策略.py 市值因子 + 动量波动率过滤



融合:

- 7-因子选股+中枢网格.py 因子选标的 + 中枢做网格
- 8-ML增强多因子.py LightGBM优化因子权重

Summary

Thinking: 学了这么多策略, 实际中怎么选?

市场环境	推荐策略	原因
震荡市, 单标的	中枢网格(脚本2)	中枢内收租, 风险可控
震荡市, 多标的	因子选股+网格(脚本7)	因子选出最适合震荡的标的
趋势市	海龟/缠论三买(上节课)	网格在趋势市没优势
不确定	混合策略(脚本3)	中枢内网格, 出中枢趋势
追求alpha	ML选股(脚本8)	LightGBM发现非线性因子组合
简单稳健	多因子打分(脚本5)	8因子等权, 月度调仓

Summary (量化投资完整体系)

- 选股层: 多因子打分 + 小市值轮动 + ML增强
- 择时层: 缠论中枢(震荡) + 海龟突破(趋势) + 网格(区间)
- 风控层: ATR仓位管理 + 中枢止损(ZG/ZD) + 网格出界停止
- 增强层: LightGBM信号过滤 + 特征工程 + Walk-Forward验证

Summary (网格交易的哲学)

Thinking: 海龟说跟随趋势, 缠论说识别结构, 网格说什么?

网格说: 不预测方向, 只在区间内收租。

- 海龟的哲学: 市场有惯性, 涨了还会涨
- 缠论的哲学: 市场有结构, 结构决定买卖点
- 网格的哲学: 市场会震荡, 震荡就有差价

三种策略, 三种市场观, 覆盖了三种市场状态:

- 趋势市 → 海龟/缠论
- 震荡市 → 网格
- 不确定 → 多因子选股 + 自适应切换

Summary (量化投资完整体系)

网格+多因子的最大价值, 不是预测市场会涨还是跌, 而是:

1. 因子选股: 科学地选出值得交易的品种
2. 中枢网格: 在震荡区间内机械收租
3. 风险控制: 出中枢即停, 不逆势加仓
4. ML增强: 让数据告诉你因子怎么组合最好

=> 把投资从凭感觉变成有规则, 从拍脑袋变成数据驱动。

打卡：策略融合与ML增强



针对你的股票池（也可以是全量的大A股票），进行：

- 用网格适配因子打分, 选出适合做网格的标的
- 对选出的标的运行中枢网格回测
- 构建13维特征, 运行LightGBM Walk-Forward回测
- 分析特征重要性, 和缠论ML/海龟ML的特征重要性对比

Thinking：为你的投资组合设计策略方案：

- 哪些标的适合网格？
- 哪些标的适合趋势跟踪？
- 如何用因子选股优化标的选择？



Thank You
Using data to solve problems